

UNIT 4 ASSIGNMENT REPORT

Unit 4: Non-Linear & Advanced Data Structures – Report

Overview

This unit focuses on non-linear data structures such as trees, graphs, heaps, and advanced structures like hash tables, tries, and bloom filters. These structures are widely used in real-world applications like search engines, databases, and networking.

5.1 Experiment 20: BST Insert / Search / Inorder

Objective

To implement basic operations in a Binary Search Tree (BST).

Key Concepts

- Left subtree → values smaller than root
- Right subtree → values greater than root

Operations

- **Insert:** Place element at correct position
- **Search:** Traverse left/right based on value
- **Inorder Traversal:** Produces sorted output

Complexity

- Average: $O(\log n)$
 - Worst: $O(n)$
-

5.2 Experiment 21: BST Delete (All Cases)

Objective

To handle deletion in BST.

Cases

1. **Leaf node** → directly remove
2. **One child** → replace with child
3. **Two children** → replace with inorder successor

Learning

Deletion is the most complex BST operation.

5.3 Experiment 22: Heap / Priority Queue

Objective

To implement heap and understand priority queues.

Key Concepts

- **Max Heap:** Parent $\geq$ children
- **Min Heap:** Parent $\leq$ children

Applications

- Task scheduling
- Dijkstra's algorithm
- OS process management

Complexity

- Insert: $O(\log n)$
 - Delete: $O(\log n)$
-

5.4 Experiment 23: Graph Build (Adjacency List)

Objective

To represent graphs efficiently.

Representation

- **Adjacency List** preferred over matrix for sparse graphs

Advantages

- Saves space
 - Efficient traversal
-

5.5 Experiment 24: BFS Traversal

Objective

To explore graph level by level.

Working

- Uses **Queue**
- Visits nearest nodes first

Applications

- Shortest path (unweighted graph)
 - Social networks
-

5.6 Experiment 25: DFS Traversal

Objective

To explore graph deeply.

Working

- Uses **Stack / Recursion**
- Goes deep before backtracking

Applications

- Cycle detection
 - Topological sort
-

5.7 Experiment 26: Hash Table (Separate Chaining)

Objective

To implement hashing with collision handling.

Key Concepts

- Hash function maps key $\rightarrow$ index
- Collisions handled using **linked list (chaining)**

Complexity

- Average: **$O(1)$**
 - Worst: **$O(n)$**
-

5.8 Experiment 27: Trie (Prefix Tree)

Objective

To store and search strings efficiently.

Key Features

- Each node represents a character
- Fast prefix search

Applications

- Autocomplete
 - Spell checking
-

5.9 Experiment 28: Bloom Filter

Objective

To understand probabilistic data structures.

Key Concepts

- Uses multiple hash functions
- Space-efficient

Properties

- No false negatives
- Possible false positives

Applications

- Databases
 - Web caching
-

5.10 Unit 4 Assignment (Student Work Summary)

Tasks Completed

- Implemented BST operations
- Worked with heaps and priority queues
- Built and traversed graphs
- Implemented hashing and tries
- Studied bloom filter concept

Learning Outcomes

- Understanding of non-linear structures
 - Efficient data storage and retrieval
 - Real-world application knowledge
-

Conclusion

This unit builds a strong foundation in advanced data structures. It emphasizes:

- Efficient searching and storage
- Graph traversal techniques
- Real-world problem-solving

These concepts are essential for competitive programming, system design, and software development.
